

Pet Adoption & Fostering Network

Project report: design, implementation, and database concepts

Project scope

A relational database and Flask web application for managing shelters, animals, adopters, fostering, healthcare follow-up, and adoption workflows.

Prepared from the project source files and the database loaded into MySQL 8.0.

1. Executive summary

The Pet Adoption & Fostering Network is an academic DBMS project that organizes rescue and adoption operations around a shared MySQL database. It replaces disconnected lists with linked records, integrity rules, reusable views, and database routines. A Flask application provides pages for browsing records and finding animals that match adopter preferences.

The database was loaded into the local MySQL 8.0 server as **pet_adoption_network**. The imported sample dataset contains five shelters, 25 animals, 15 adopters, ten foster homes, 15 medical records, and 15 adoption applications. The schema also includes fostering, home visits, adoption records, and medical alert records.

2. Goals and scope

- Represent the main entities and their relationships with keys, constraints, and foreign keys.
- Demonstrate SQL data definition and manipulation, joins, nested queries, aggregates, views, indexes, and transaction control.
- Automate important integrity checks with triggers and provide reusable operations through stored procedures.
- Expose common tasks through a local Flask interface, including record browsing and adopter-to-animal matching.

This is a teaching project. It does not implement production-grade authentication, payments, identity verification, or deployment hardening.

3. Data model

The model follows the operational lifecycle: shelters list animals; adopters submit applications for animals; applications can have home visits; fostering connects animals with foster homes; medical records track care; completed applications are recorded as adoptions.

Relationship	Meaning
Shelters 1 to many Animals	A shelter can list many animals; an animal may have no current shelter.
Animals 1 to many Medical Records	Each health visit belongs to one animal.
Adopters 1 to many Applications	An adopter can apply for multiple animals.
Applications 1 to many Home Visits	A case can record visits and recommendations.
Animals many to many Foster Homes	The FOSTERING table records each placement and its dates/status.
Approved Application to Adoption Record	A completed adoption is linked to its originating application.

Core relations

Area	Tables	Key design choices
People and locations	shelters, adopters, foster_homes	Primary keys; unique contact fields; capacity/age checks.
Animal care	animals, medical_records, medical_followup_alerts	Animal status, vaccination, health details, check-up dates.
Placement	fostering	Associative relation with start/end dates, status, and capacity checks.
Adoption	adoption_applications, home_visits, adoption_records	Application decision, home assessment, and completed outcome.

4. Database implementation

The implementation separates schema, data, and behavior into ordered SQL files. The schema creates the database and relational structures. Sample data populates representative records. Views present joined data for common screens. Triggers protect selected rules when rows change. Procedures package multi-step operations.

Component	What it contributes
schema.sql	Database, ten base tables, primary/foreign keys, enumerated statuses, checks, unique constraints, and indexes.
sample_data.sql	Example records for shelters, animals, foster placements, healthcare, applications, visits, and adoptions.
views.sql	Available animals, medical follow-up state, and an application summary with adopter/animal/visit details.
triggers.sql	Medical alert creation, adoption status synchronization, and foster capacity validation.
procedures.sql	Atomic adoption approval, overdue alert generation, and an availability report.
app.py + templates	Flask routes and pages for dashboard, animals, shelters, adopters, medical follow-up, applications, adoptions, and matching.

5. Relational design and integrity

The schema uses identifiers as primary keys and foreign keys to prevent orphaned references. Unique constraints protect such values as phone numbers and application-to-adoption links. Check constraints enforce rules such as adult adopter age and positive capacity. Enum columns constrain statuses to known values.

The design aims for normalized relations: attributes describe their own entity, repeating groups are represented as separate rows, and relationships are linked by keys. The fostering relation resolves a many-to-many relationship. Independent multi-valued preferences could be split into additional relations in a future extension to represent fourth normal form more fully.

6. DBMS concepts demonstrated

Concept	Project example
SQL and relational queries	Filtering, joins, grouping, aggregates, subqueries, set operations, and adopter matching.
Indexes	Status, species/size, application status, medical follow-up, and foster lookups.
Views	Reusable read models for animals, follow-up, and application summaries.
Triggers and procedures	Event-driven integrity rules and reusable database operations.
Transactions	Adoption approval validates prerequisites and commits or rolls back as one unit.
Query optimization	The project materials discuss query cost and plans; MySQL EXPLAIN can be used to inspect actual plans.

The academic materials also cover B+ trees, hashing, query optimization, isolation, locking, deadlocks, two-phase commit, and recovery. These topics are explained conceptually; the live application does not implement a distributed transaction coordinator.

7. Limitations and future work

The project is a local academic demonstration. It should be extended with role-based access, validated user input and authentication, audit history, automated notifications, backup/restore procedures, and a deployment configuration before use with real shelter operations.

8. Conclusion

The project demonstrates how a relational model, integrity constraints, SQL automation, transactions, and an application layer can support a practical workflow. The loaded database provides a working basis for exploring those ideas with sample adoption and care data.

Source note: prepared from the included README, SQL scripts, Flask application, report notes, and the successful local import. Database counts are the values verified after import.